

M4: Test three ways to repair typo queries

Beyond Benchmarks: Robustness for Real-World Information Systems

Bum Jun Kim, The University of Tokyo, CIKM 2026

7 November 2026 · San Pietro in Vincoli Campus, Sapienza University of Rome · Rome, Italy

Learning objective: Test text normalization, spelling correction, and a query-vector adapter. Check whether each repair helps typo queries and harms correctly written queries.

Run this notebook on its own with Restart Kernel → Run All. It uses saved SciFact results. See the README for setup and the reference for technical details.

```
# Load the libraries, SciFact data, and saved model results.
from pathlib import Path
import sys
ROOT = next(p / "resources" for p in [Path.cwd(), *Path.cwd().parents] if (p / "resources" /
"tutorial_utils").is_dir())
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
%matplotlib inline
from tutorial_utils.presentation import install_plain_display
install_plain_display()
from tutorial_utils import load_tutorial_context
from tutorial_utils import improvement as imp
from tutorial_utils.interventions import input_normalization, build_vocabulary, vocab_spell_correction

MODE = "cached"
ctx = load_tutorial_context(mode=MODE, seed=42)
print(f"SciFact: {len(ctx.corpus):,} documents, {len(ctx.test_ids)} test queries, cached results")
```

```
SciFact: 5,183 documents, 300 test queries, cached results
```

Can a repair recover the evidence?

A useful repair helps typo queries and preserves good results on correctly written queries.

Method	What it changes
Input normalization	Letter width, case, and spacing
Vocabulary correction	Spelling, using words from the papers
Repair adapter	The query vector

Interpretation: The first two methods edit the text, while the adapter leaves the text unchanged and adjusts its vector.

Compare every repair with `Original search`, which is dense search without repair. Use the same input for each comparison. The papers and encoder stay fixed. Reranking is off.

Experiment Settings

`severity` is the maximum character edits per query, from 0 to 3. The default is 1. Zero leaves the input unchanged.

After changing it, select the code cell below and use Run All Below. This updates the examples, scores, and failure cases.

```
# Set the maximum number of character edits per query.
severity = 1  # 0: unchanged input; 1–3: maximum character edits per query.
```

Input normalization: Standardize the text

Normalization:

- Converts full-width letters to standard letters.
- Makes letters lowercase.
- Removes extra whitespace.

For example, `ＭＵＳＣＬＥ　pain` becomes `muscle pain`. It does not fix spelling: `musckle` keeps its extra `k`.

```
# Standardize letter width, case, and spacing in an example.
text = "ＭＵＳＣＬＥ　pain"
print("Before:", text)
print("After: ", input_normalization(text))
```

```
Before: ＭＵＳＣＬＥ　pain
After:  muscle pain
```

Interpretation: Only the form of the text changed: the full-width capitals became ordinary lowercase letters and the extra spaces became one space, while the letters of each word stayed the same.

Vocabulary correction: Fix unknown words

Build a word list from the papers. Then check each eligible query word:

1. Keep it if it is in the word list.
2. Otherwise, find words one character edit away.
3. Choose the most frequent match that meets the rules. Keep the input if none qualifies.

For example, `musckle` becomes `muscle` by deleting `k`. A correctly spelled `muscle` stays unchanged because it is already in the list. The reference gives the exact rules.

```
# Build a vocabulary from the papers and correct an example typo.
vocabulary = build_vocabulary(ctx.corpus)
word = "musckle"  # Typo input; "muscle" is the corresponding correct spelling.
print("Before:", word)
print("After: ", vocab_spell_correction(word, vocabulary))
```

```
Before: musckle
After:  muscle
```

Interpretation: `musckle` is not in the papers' word list, so correction replaced it with `muscle`, the most frequent listed word one character edit away, which restores the intended word.

What text does the encoder read?

Apply both text repairs to the full claim. Check which one fixes the typo.

```
# Compare the original query, its typo version, and both text repairs.
input_example = imp.show_inputs(ctx, severity=severity, query_id="1303")
```

Selected test query, maximum edits: 1

Original query: Tirasemtiv has no effect on fast-twitch muscle.

After typos: Tirasemtiv has no effect on fast-twitch musckle.

After normalization: tirasemtiv has no effect on fast-twitch musckle.

After vocabulary correction: Tirasemtiv has no effect on fast-twitch muscle.

Interpretation: Only vocabulary correction restores the original claim; normalization only lowercases the text and keeps musckle.

Repair adapter: Adjust the query vector

The adapter learns from pairs of typo and original training queries. It learns to move typo vectors closer to their original vectors. Validation queries help choose how strongly to limit the adjustment.

It applies the same learned adjustment to original and typo queries. The text, encoder, and paper vectors stay fixed.

The next example checks whether this adjustment recovers the evidence. The reference explains training, validation, and duplicate-query checks.

```
# Prepare repair results and show the adapter's effect on the example query.
lesson_result = imp.prepare_lesson(ctx, severity=severity)
adapter_example = imp.show_adapter_example(ctx, lesson_result, query_id="1303")
```

Selected test query, maximum edits: 1

Input to both searches: Tirasemtiv has no effect on fast-twitch musckle.

The text stays unchanged. The adapter adjusts the 384-number query vector.

Without adapter

First paper: Efficacy and tolerability of amitriptylinoxide in the treatment of chronic tension-type headache: a multi-centre controlled study.

Known relevant papers in top 20: 0 / 1.

With adapter

First paper: Amitriptyline is effective in chronic but not in episodic tension-type headache: pathogenetic implications.

Known relevant papers in top 20: 0 / 1.

Both searches miss the known evidence in this example. The next table compares all 300 queries.

Interpretation: The adapter changes the first paper but does not bring the known evidence back into the top 20, so it does not repair this query.

Search quality on all 300 test queries

Both score columns show mean nDCG@10. Higher is better.

- `Original input`: does the repair preserve quality on correctly written queries?
- `After typos`: does it improve quality on typo queries?

Compare each repair with `Original search` in the same column.

```
# Compare each repair's search quality on original and typo queries.
comparison_table = imp.show_lesson_comparison(lesson_result)
```

nDCG@10 (higher is better), 300 test queries, maximum edits: 1

Method	Original input	After typos
Original search	0.6451	0.6220
Input normalization	0.6451	0.6220
Vocabulary correction	0.6432	0.6437
Repair adapter	0.6539	0.6229

Vocabulary correction scores highest on the after-typo inputs (0.6437), compared with 0.6220 for original search.

Interpretation: Vocabulary correction recovers almost all of the typo loss (0.6437 after typos, close to original search's 0.6451 before typos) at a small cost on correctly written queries (0.6451 → 0.6432), while input normalization changes no score and the repair adapter mainly raises original-query scores.

Where does each repair fail?

For each method, show the query where it causes the largest nDCG@10 loss. Check both original and typo inputs.

If it causes no loss, show the largest typo loss it fails to fix. Read the query text and evidence ranks to see what went wrong.

Normalization leaves the typo

This encoder already ignores case and extra spaces, so normalization leaves these scores unchanged. The example shows the largest typo loss it fails to fix.

```
# Show a query whose typo damage remains after normalization.
normalization_failure = imp.show_failure_case(ctx, lesson_result, "input_normalization")
```

Input normalization, maximum edits: 1

Original query: Tirasemtiv has no effect on fast-twitch muscle.

After typos: Tirasemtiv has no effect on fast-twitch musckle.

After normalization: tirasemtiv has no effect on fast-twitch musckle.

Typo: muscle → musckle. Normalization keeps the misspelling.

Original search, original query

First paper: Activation of fast skeletal muscle troponin as a potential therapeutic approach for treating neuromuscular diseases

Known relevant papers in top 20: 1 / 1, at rank 1. nDCG@10: 1.0000.

Input normalization, after typos

First paper: Efficacy and tolerability of amitriptylinoxide in the treatment of chronic tension-type headache: a multi-centre controlled study.

Known relevant papers in top 20: 0 / 1. nDCG@10: 0.0000.

In this run: Normalization gave the same nDCG@10 as the original search for all 300 queries. Typos lowered nDCG@10 for 50 queries; normalization returned none of them to the original-query score.

Interpretation: Normalization is harmless but does not help against these typos: musckle still reaches the encoder, so nDCG@10 stays at 0.0000 instead of 1.0000.

Spelling correction changes a correct word

A correct word can be missing from the papers' word list. The example shows it being replaced by a word with a different meaning. The output includes both words' counts in the papers.

```
# Show the query most harmed by vocabulary spelling correction.
correction_failure = imp.show_failure_case(ctx, lesson_result, "vocab_spell_correction")
```

Vocabulary correction, original input

Input: The risk of male prisoners harming themselves is ten times that of female prisoners.

After correction: The risk of male prisoners warming themselves is ten times that of female prisoners.

Replaced word: harming → warming. In the documents, harming appears 0 times and warming 3 times.

Original search

First paper: Self-harm in prisons in England and Wales: an epidemiological study of prevalence, risk factors, clustering, and subsequent suicide

Known relevant papers in top 20: 1 / 1, at rank 1. nDCG@10: 1.0000.

Vocabulary correction

First paper: Prison health care: a review of the literature.

Known relevant papers in top 20: 1 / 1, at rank 4. nDCG@10: 0.4307.

In this run: On original input, correction lowered nDCG@10 for 1 query and raised it for 1 compared with original search. After typos, it lowered nDCG@10 for 19 queries and raised it for 48. In 17 of those 19 queries, correction restored the original wording exactly. The typo versions had scored higher than the originals.

Of the 300 typo words, correction restored 263, left 31 unchanged, and replaced 6 with a different word. For example, laher → later, but the original word was layer.

Interpretation: Because harming never appears in the papers, correction treats it as a typo and changes it to warming, which moves the relevant paper down from rank 1 to rank 4 on this correctly written claim.

The adapter moves evidence down

A learned vector adjustment can harm a correctly written query. The example shows the adapter's largest loss. Compare the relevant paper's rank before and after the adjustment.

```
# Show the query most harmed by the repair adapter.
adapter_failure = imp.show_failure_case(ctx, lesson_result, "repair_adapter")
```

Repair adapter, original input

Input to both searches: Tirasemtiv has no effect on fast-twitch muscle.

The text stays unchanged. The adapter adjusts the 384-number query vector.

Original search

First paper: Activation of fast skeletal muscle troponin as a potential therapeutic approach for treating neuromuscular diseases

Known relevant papers in top 20: 1 / 1, at rank 1. nDCG@10: 1.0000.

Repair adapter

First paper: Exercise blood flow patterns within and among rat muscles after training.

Known relevant papers in top 20: 1 / 1, at rank 9. nDCG@10: 0.3010.

In this run: On original input, the adapter lowered nDCG@10 for 32 queries and raised it for 36 compared with original search. After typos, it lowered nDCG@10 for 33 queries and raised it for 38.

Interpretation: The adapter changes only the vector of this correctly written query, not its text, and that shift moves the relevant paper down from rank 1 to rank 9.

Lesson Summary

Check three things: does the repair fix the typo, improve search after typos, and preserve quality on original queries?

Normalization leaves these typos unchanged. Spelling correction and the adapter help some queries but harm others. Check the largest individual losses as well as the average.

Reference: Training and evaluation

Text-repair rules

Normalization uses Unicode NFKC to standardize character forms, case folding to ignore case, and single spaces to replace extra whitespace.

Vocabulary correction counts words in the papers. It can change a word only when the word:

- Is lowercase and contains only letters.
- Has at least five letters.
- Is missing from the word list.

A replacement must be one character insertion, deletion, or substitution away. It must appear at least three times in the papers. Choose the most frequent match; break ties alphabetically. If no match qualifies, keep the word. Capitalized words and words containing numbers stay unchanged.

The failure check records whether each typo was restored, left unchanged, or replaced by another word.

Adapter training

The query vector z has length 1. Learn a 384×384 matrix A, add Az, and scale the result back to length 1:

$$z' = \mathbf{normalize}(z + Az), \qquad A^* = \arg\min_A \sum_{q \in Q_{\text{train}}} \|(I + A)z_{\tilde{q}} - z_q\|_2^2 + \lambda \|A\|_F^2.$$

The first term makes adjusted typo vectors closer to original vectors. The second limits the adjustment's size. A larger λ limits it more, but does not guarantee better search. $A = 0$ means no adjustment. Paper vectors and encoder weights stay fixed.

Remove duplicate query text across splits. Keep the test copy first, then validation, then training. This removes one validation query and two training queries. The remaining sets have 645 training, 161 validation, and 300 test queries.

Train with one-edit query pairs. Choose λ from $(0.1, 1, 10, 100)$ using validation typo nDCG@10. Break ties by original-query validation score, then by smaller λ. Validation uses the selected severity. Test scores are used only after this choice. The default chooses $\lambda = 0.1$.

M6 keeps adapter training and validation at one edit while varying only the test severity. Both modules use the same adapter at the default `severity=1`; at other settings, M4 can select a different adapter.

Three score differences

$R_m(q, s)$ is method m's nDCG@10 at severity s. Method b is original dense search. For every difference below, higher is better and negative means a loss.

Quantity	Definition	Comparison
Effect of typos	$R_m(q, s) - R_m(q, 0)$	Same method, different input
Repair gain on typo input	$R_m(q, s) - R_b(q, s)$	Same typo input, different method
Repair gain on original input	$R_m(q, 0) - R_b(q, 0)$	Same original input, different method

Interpretation: To measure typo damage, change the input and keep the method; to measure repair benefit, change the method and keep the input.

Calculate each difference for matching query IDs, then average. Optional 95% intervals show how the mean may vary across samples of queries. Use 500 bootstrap samples: draw query IDs with replacement and keep each query's two scores together. The model outputs stay fixed.

Failure-case selection

Choose the largest loss caused by a repair: maximize $R_b(q, c) - R_m(q, c)$ over test queries and $c \in \{0, s\}$. Break ties by original input, then test order.

If the repair causes no loss, choose the largest typo loss it leaves unfixed: $R_b(q, 0) - R_m(q, s)$.

Optional diagnostics:

```
imp.show_selection(ctx, lesson_result["candidate"])
imp.show_effects(lesson_result)
```

M1–M3 explain qrels, metrics, and typo rules. M6 checks whether the repair passes the rules for use.

Sources to read

- Norvig (2007, updated 2016), How to Write a Spelling Corrector.
- Zhuang & Zuccon (SIGIR 2022), CharacterBERT and Self-Teaching for Improving the Robustness of Dense Retrievers on Queries with Typos.

Next: M5: Protect RAG references from document attacks.